

임요섭 포트폴리오 (신입)

E-Mail : yoxup.lim@gmail.com

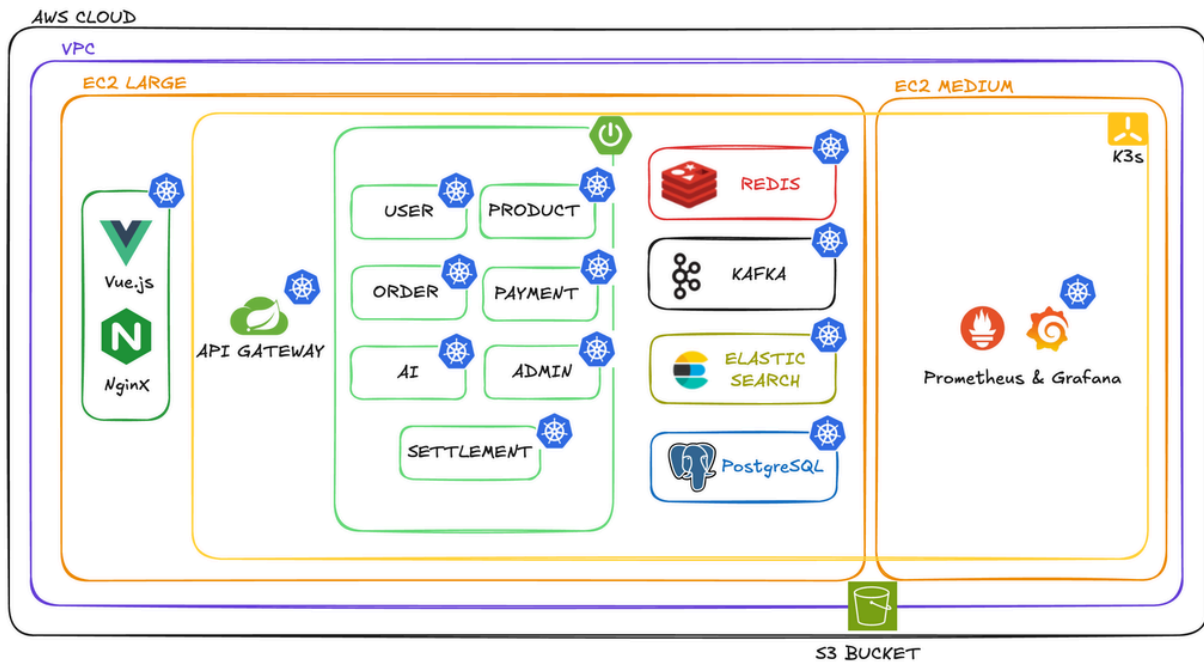
H.P : 010-9139-4102

Blog : <https://bdisappointed.tistory.com>

Github : <https://github.com/xub2>

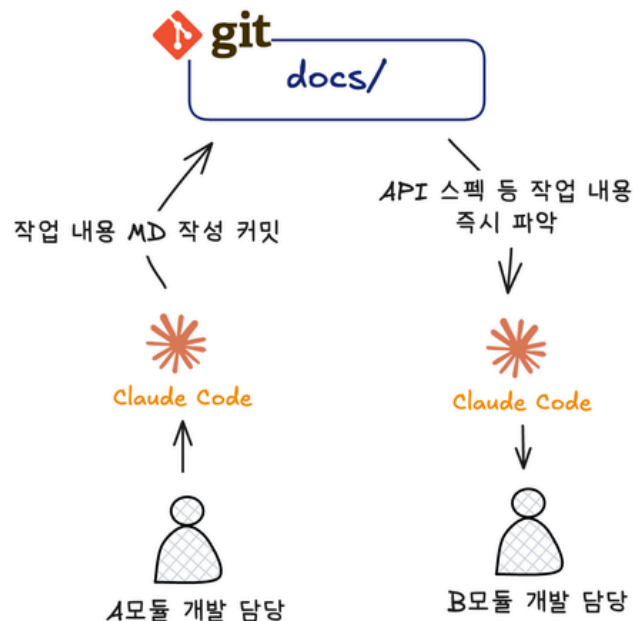
Project Portfolio #1 : 원데이 클래스 예약 플랫폼 : 잡아 클래스 (Jaba Class)

Project Architecture



Technical Challenges

1. MD 기반 문서화 문화 구축과 Claude Code 컨텍스트 활용



- 배경

- MSA 7개 서비스를 팀원들이 나눠 개발하다 보니 **두 가지 문제 존재**
 - 다른 팀원의 모듈을 파악하려면 직접 물어보거나 코드를 처음부터 읽어야 했고, 이 과정이 반복될수록 개발 집중도 및 생산성 저하
 - 프로젝트에 Claude Code를 도입했지만 세션이 초기화될 때마다 컨텍스트를 다시 설명해야 하는 비효율 존재

- 해결 방향

- 별도의 도구를 도입하는 대신, **Git 레포지토리 안에 docs/ 폴더를 만들어 서비스별 설계 결정, API 명세, 장애 대응 흐름을 MD로 직접 정리**
 - API Gateway, User, Product, Order, Payment, Settlement, Admin, AI, ERD, Kafka, Elasticsearch, k3s, CI/CD 총 13개 카테고리 구조화
 - 팀원 누구나 PR 전 관련 문서를 참고하거나 업데이트하는 문화 정착

Name	Last commit message	Last commit date
..		
00_api-spec	docs: 전체 문서 구조 개편 및 신규 문서 추가	2 months ago
00_auth	chore: 인증 관련 md 내용 수정	last month
01_user	chore: 인증 관련 md 내용 수정	last month
02_product	docs: 전체 문서 구조 개편 및 신규 문서 추가	2 months ago
04_order	docs: 전체 문서 구조 개편 및 신규 문서 추가	2 months ago
05_payment	docs: 전체 문서 구조 개편 및 신규 문서 추가	2 months ago
06_settlement	docs : settlement md 파일 최신화	2 months ago
07_admin	docs: 전체 문서 구조 개편 및 신규 문서 추가	2 months ago
08_ai	docs: 전체 문서 구조 개편 및 신규 문서 추가	2 months ago
09_ERD	docs: 전체 문서 구조 개편 및 신규 문서 추가	2 months ago
10_kafka	docs: 전체 문서 구조 개편 및 신규 문서 추가	2 months ago
11_elasticsearch	docs: 전체 문서 구조 개편 및 신규 문서 추가	2 months ago
12_k3s	feature: gateway rule 적용 workflow md 추가	2 months ago
13_CICD	fix: 내용 수정	last month
INDEX.md	fix: INDEX.md 수정: api-gateway 에 대한 내용 수정	last month

- 효과

- 팀원들이 같은 질문을 반복하지 않게 되어 개발 집중도 향상
- 기록이 쌓일수록 Claude Code의 컨텍스트 품질도 높아져 팀 전체 개발 생산성 향상

2. 서버 리소스 이슈 #1 — Elasticsearch 도입 결정 과정

• 배경

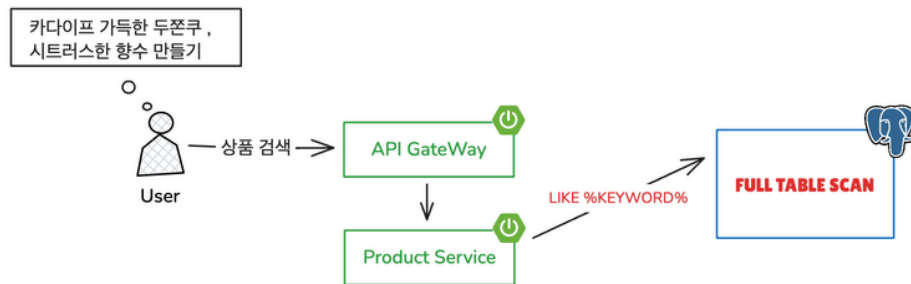
- t3.large 위에서 7개 서비스 + Redis, Kafka, DB 운영 → 추가 인프라 도입은 신중하게 고려해야 함
- PostgreSQL pg_trgm GIN 인덱스 확장 존재 → ES 없이 DB 레벨에서 먼저 검증

• 문제 원인

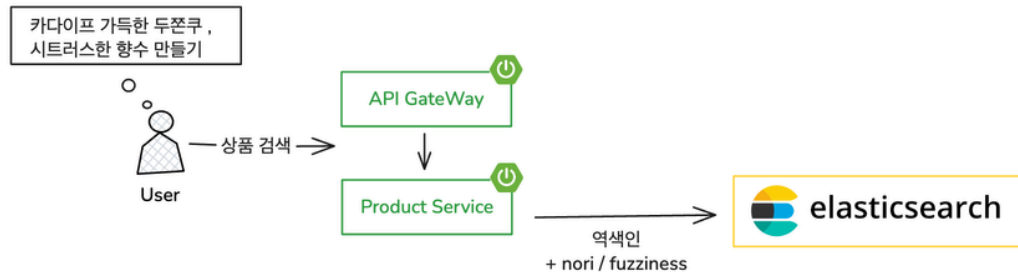
- 유명 셀러 클래스 오픈 시 단시간 검색 트래픽 집중 스파이크 패턴 발생 상황 가정
 - 기존 JPA LIKE %keyword% 방식 → 앞 와일드카드로 Full Table Scan 동작
 - 데이터 규모가 커질수록 선형(O(N)) 비용 증가

• 아키텍처

● 기존 방식 (JPA LIKE FULL SCAN)



● 개선 방식 (ElasticSearch 도입)



• 해결 과정

- GIN 적용 결과 p95 1,879ms → 1,654ms (12% 개선), 극한 부하 시 실패 14건 발생
 - 검색 실패 = 예약 기회 손실 → 읽기 요청이라도 실패 허용 불가
- GIN 한계 수치로 확인 후 ES 도입 결정, nori 형태소 분석기로 한국어 전문 검색 지원

• 테스트

- k6로 1,000건 / 10만 건 / 100만 건 3단계 부하 테스트 수행
- ES vs DB 풀스캔 vs DB+GIN 3가지 옵션에 대한 테스트 진행

• 결과

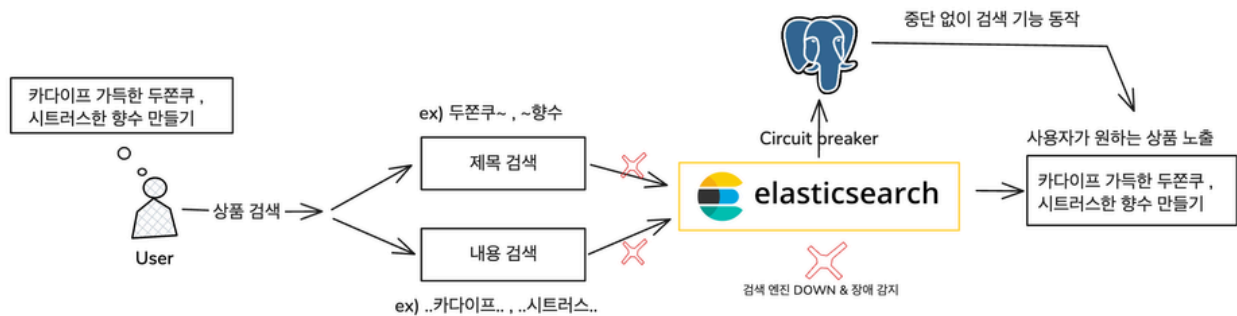
- DB 풀스캔 p95 1,879ms → DB+GIN p95 1,654ms → ES p95 1,294ms (DB 풀스캔 대비 31% 개선)
- 데이터 1,000건 → 100만 건 증가 시 DB p95 +153%, ES p95 +85% 증가로 ES의 확장 안정성 확보

3. 서버 리소스 이슈 #2 — 단일 ES 노드로 인한 SPOF 문제 해결

- 배경

- 검색 로직 전체를 ES에 위임한 구조 → ES 단일 장애 발생 시 검색 API 전체 오류로 이어지는 SPOF 문제
- t3.large 리소스 제약으로 ES 노드(파드) 추가 불가 → 인프라 확장 대신 애플리케이션 레벨에서 해결책 필요

- 아키텍처



- 해결 과정

- 검색 메서드(searchAll, searchMy)에 Resilience4j Circuit Breaker 적용
- ES 장애 감지 시 즉시 PostgreSQL 직접 조회로 fallback 전환해 검색 기능 유지
- ES 컨테이너를 강제 종료하는 카오스 테스트를 설계해 CB 상태 전이와 fallback 동작 검증

- 테스트

- 정상 구간 운영 후 ES 컨테이너 강제 종료로 실제 장애 재현
- CB가 OPEN으로 전환되며 PostgreSQL fallback으로 응답이 유지되는 과정 관찰
- ES 재시작 후 CB가 HALF_OPEN → CLOSED로 자동 복구되는 흐름 검증

- 결과

- ES 장애 중 검색 API 실패율 0% 유지 (fallback DB 조회로 응답 지속)
- ES 복구 후 CB 자동 전환 및 검색 기능 정상 동작 확인

4. 검색 기능 장애 대응 — Race Condition으로 인한 ES Dynamic Mapping 문제

- 배경

- 서버 비용 절감을 위해 매일 EC2를 껐다 켜는 운영 방식 채택
- 서버 재시작 시 Kafka에 소비되지 않은 메시지가 남아 있었고, consumer가 인덱스 초기화보다 먼저 실행

- 문제 원인

- ES는 첫 문서 색인 시 필드 타입을 자동 추론(Dynamic Mapping)하여 매핑 생성
- Kafka consumer가 인덱스 초기화보다 먼저 실행되어 nori 분석기가 빠진 상태로 디스크에 매핑 고착
- 잘못된 매핑은 디스크에 영속화되어 파드 재시작 후에도 살아남음
 - 이후 색인된 데이터들이 잘못된 매핑 위에 쌓이면서 검색 API 호출 시 한국어 형태소 분석 불가 (검색 기능 장애)

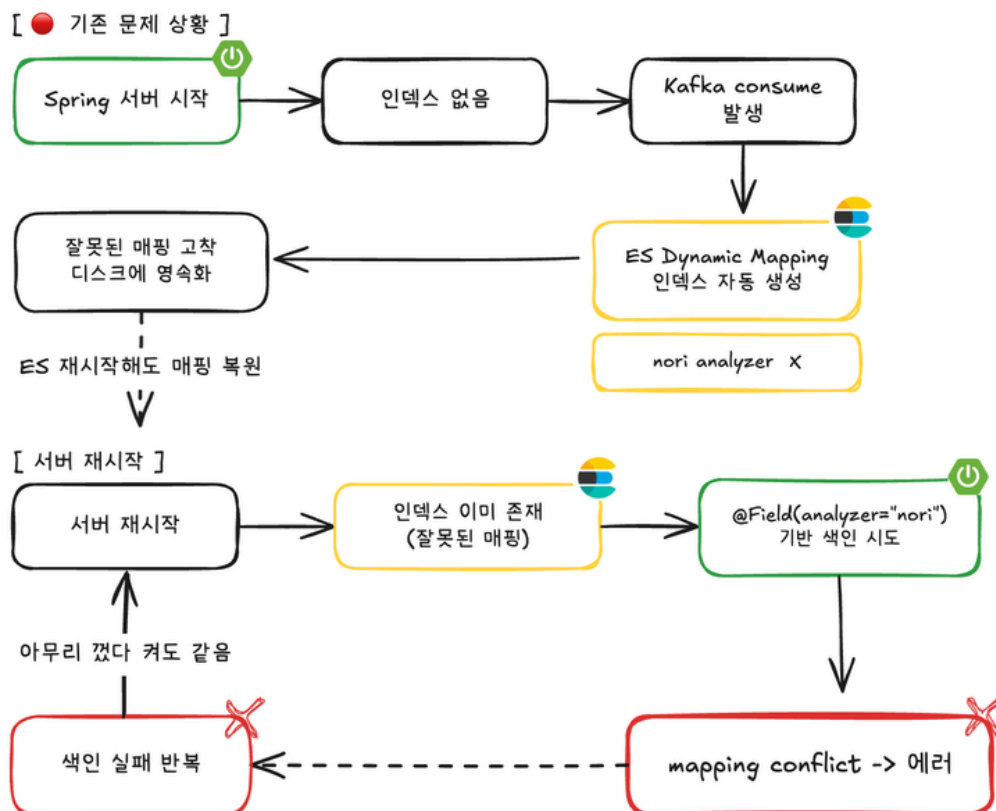
- 해결 과정

- 최초 인덱스 삭제 후 재생성으로 서비스 임시 복구
- @Document, @Setting, @Field로 nori 분석기 포함 매핑을 코드로 명시 → Dynamic Mapping 원천 차단
- @PostConstruct로 전환하여 Kafka consumer 등록 전 인덱스 초기화가 반드시 먼저 실행되도록 보장 → Race Condition 근본 원인 제거

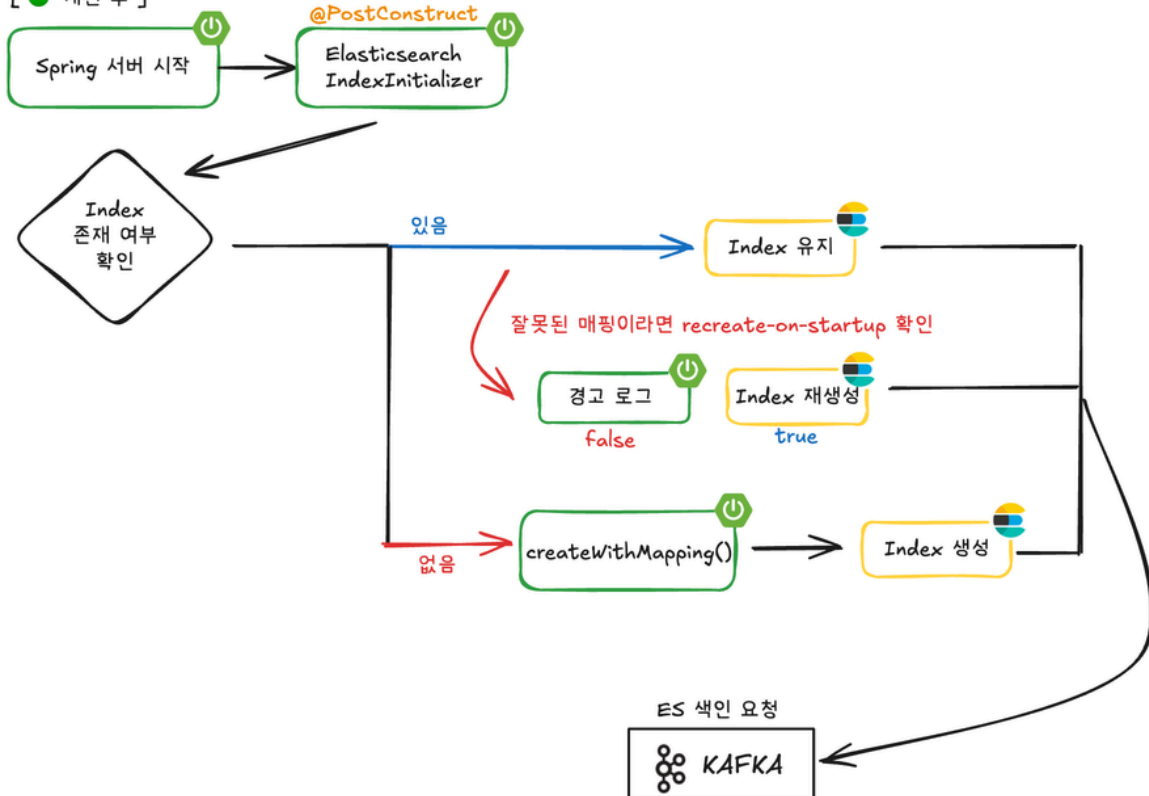
- 결과

- 인덱스 초기화 순서 보장으로 Race Condition 근본 원인 제거
- 명시적 매핑으로 nori 분석기 누락 재발 방지

- 아키텍처



[● 개선 후]



5. Admin 모듈 다중 도메인 DataSource 통합과 커넥션 풀 최적화

• 배경

- MSA 구조였지만 인프라 비용 절감을 위해 DB는 단일 PostgreSQL을 공유하는 형태로 운영
- 단일 DB를 공유하는 만큼, 각 서비스가 서로의 테이블을 직접 참조하면 도메인 경계가 무너지므로 팀 규칙상 직접 참조를 금지
- 단, Admin 서비스는 관리자 특성상 user, product, order, settlement, review 도메인에 동시 접근이 필요한 예외 케이스

• 문제 원인

- 초기엔 도메인별로 DataSource를 분리(User/Product/Order/Settlement 4개)해 구성
- 각 DataSource가 HikariCP 기본값(10개)씩 커넥션을 점유 → 이론상 최대 40개 커넥션 점유
- Admin은 트래픽이 적고 조회 위주임에도 단일 DB에서 과도한 커넥션을 점유하는 구조

• 해결 과정

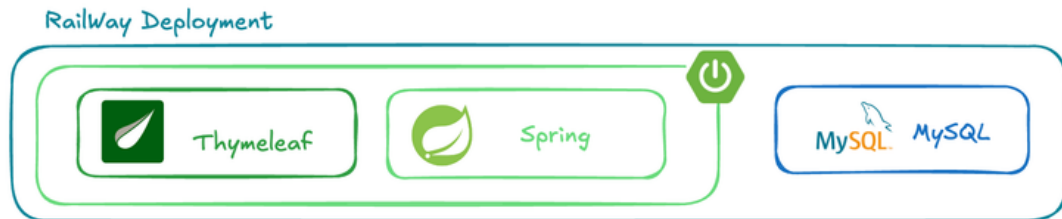
- 4개로 분리되어 있던 DataSource를 AdminDataSourceConfig 하나로 통합 (5개 도메인이 단일 풀 공유)
- TransactionManager도 4개 → 1개로 통합
- Admin 트래픽 특성을 고려해 HikariCP를 maximum-pool-size: 2 / minimum-idle: 1로 튜닝

• 결과

- Admin 모듈 최대 점유 커넥션 40개 → 2개로 축소, 단일 DB 커넥션 리소스 효율화

Project Portfolio #2 : 다니엘 청년부 행정 관리 웹 애플리케이션

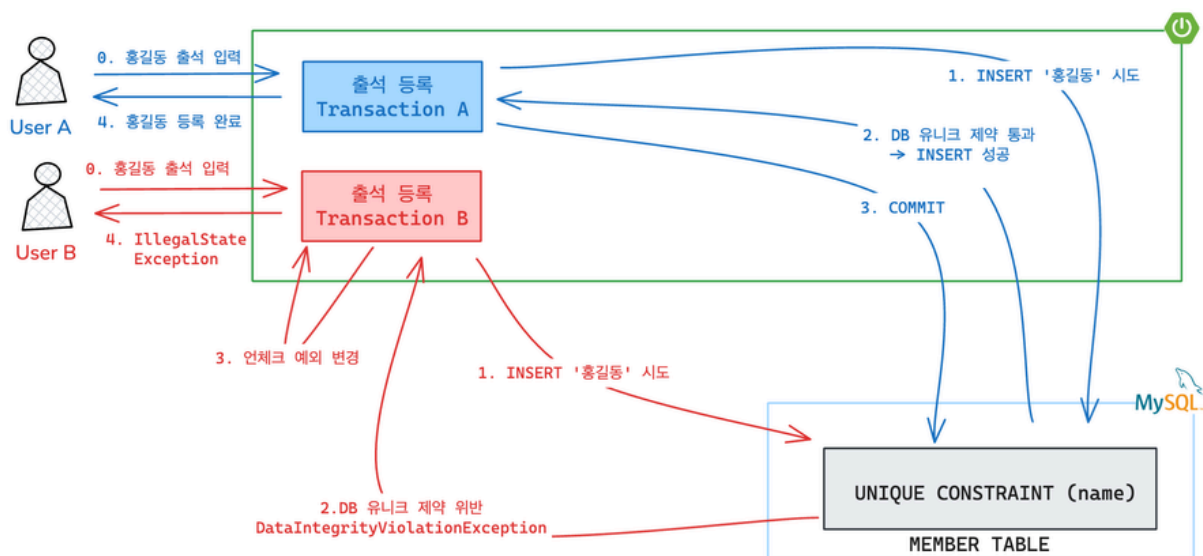
Project Architecture



Technical Challenges

1. 출석 중복 등록 문제 — DB Unique 제약 기반 동시성 처리

- 아키텍처



- 문제 원인

- 동일 인원을 여러 관리자가 동시에 출석 등록 요청할 때 같은 인원이 중복 저장되는 동시성 이슈 발생.
- 애플리케이션 단의 중복 검증 로직이 없는 상태에서 여러 트랜잭션이 동시에 DB에 INSERT를 시도하여 데이터가 중복 저장되는 현상 확인

- 해결 과정

- @Column(unique = true)로 name 컬럼에 유니크 제약 설정 → 동시 INSERT 시 InnoDB 묵시적 락으로 직렬화 처리
- DataIntegrityViolationException을 catch해 IllegalStateException으로 변환, 사용자에게 명확한 에러 응답

- 테스트

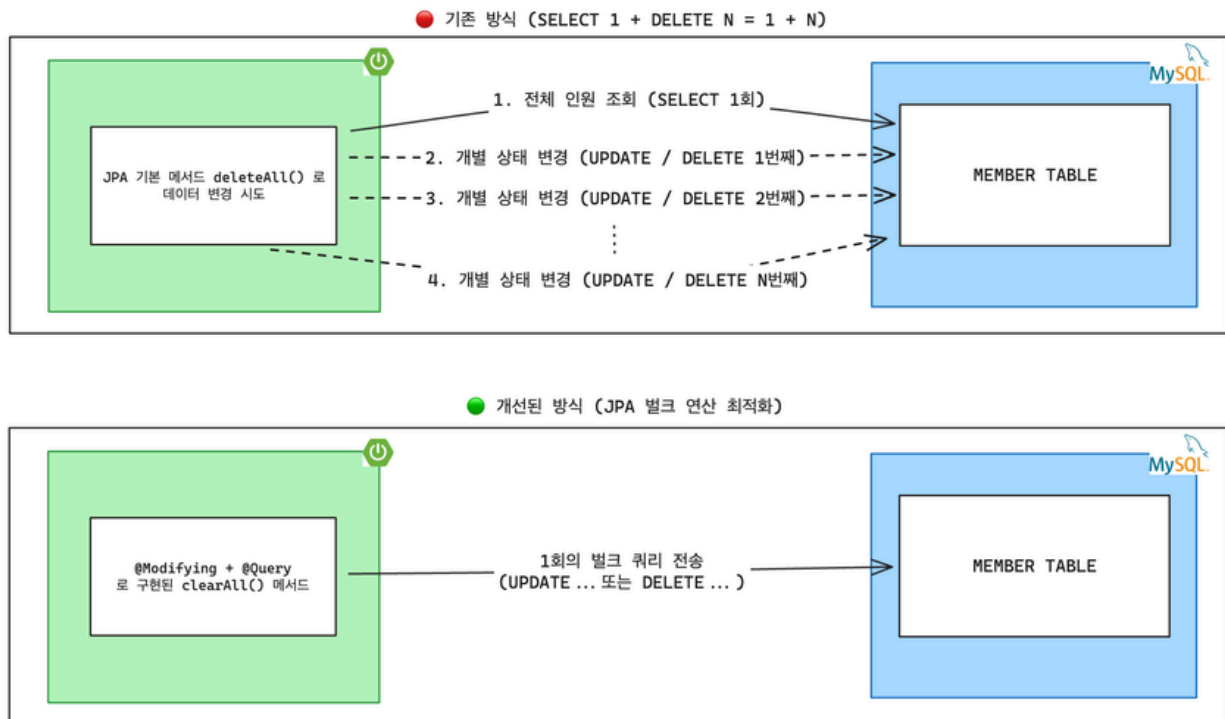
- Postman으로 동일 이름 다중 POST 요청을 동시 전송해 Race Condition 재현 → 서버 로그와 DB 조회로 정합성 검증

- 결과

- 동시 요청이 몰려도 단 1건만 저장, 데이터 중복률 0%
- 동명이인은 "이름 뒤 A 붙여 입력" 안내로 구분 등록 유도 → 정상 인원 누락 방지

2. 전체 인원 일괄 갱신 성능 개선, JPA 벌크 연산 도입

• 아키텍처



• 문제 원인

- 매주 전체 인원의 상태(목장 배정 등)를 일괄적으로 초기화하는 기능에서 성능 저하 우려 발견
- JPA가 기본 제공하는 엔티티 삭제/수정 메서드를 사용할 경우, 먼저 대상 엔티티를 모두 조회(SELECT 1회)한 후 각 엔티티마다 개별적으로 쿼리(UPDATE/DELETE N회)가 발생하는 1+N 문제 발생
- 현재 규모(30 ~ 40명 내외)를 넘어 향후 대규모 트래픽 환경이 되었을 때, 1+N 쿼리로 인한 불필요한 네트워크 I/O 급증이 시스템 성능 저하로 이어질 가능성을 고려하여 사전 최적화 진행

• 해결 과정

- 1+N 문제를 근본적으로 해결하기 위해, 영속성 컨텍스트(1차 캐시)를 거쳐 개별적으로 처리하는 방식 대신 데이터베이스에 직접 단일 쿼리를 전달하는 JPA 벌크 연산 도입
- MemberRepository 인터페이스에 전체 데이터를 한 번에 수정/삭제하는 커스텀 JPQL 작성**
- 벌크 연산은 영속성 컨텍스트를 거치지 않고 DB에 직접 쿼리를 실행하므로, 컨텍스트에 남아있던 변경분이 벌크 연산에 누락되지 않도록 @Modifying(flushAutomatically = true)로 실행 전 flush를 보장

• 결과

- 기존 1 + N회 발생하던 쿼리 실행 횟수를 단 1회로 단축**
- 대량의 데이터 갱신 시 발생할 수 있는 메모리 과부하 및 트랜잭션 타임아웃 위험을 사전에 차단하여 시스템 안정성 확보

3. 다중 조건 기반 랜덤 소그룹 배정 알고리즘 구현

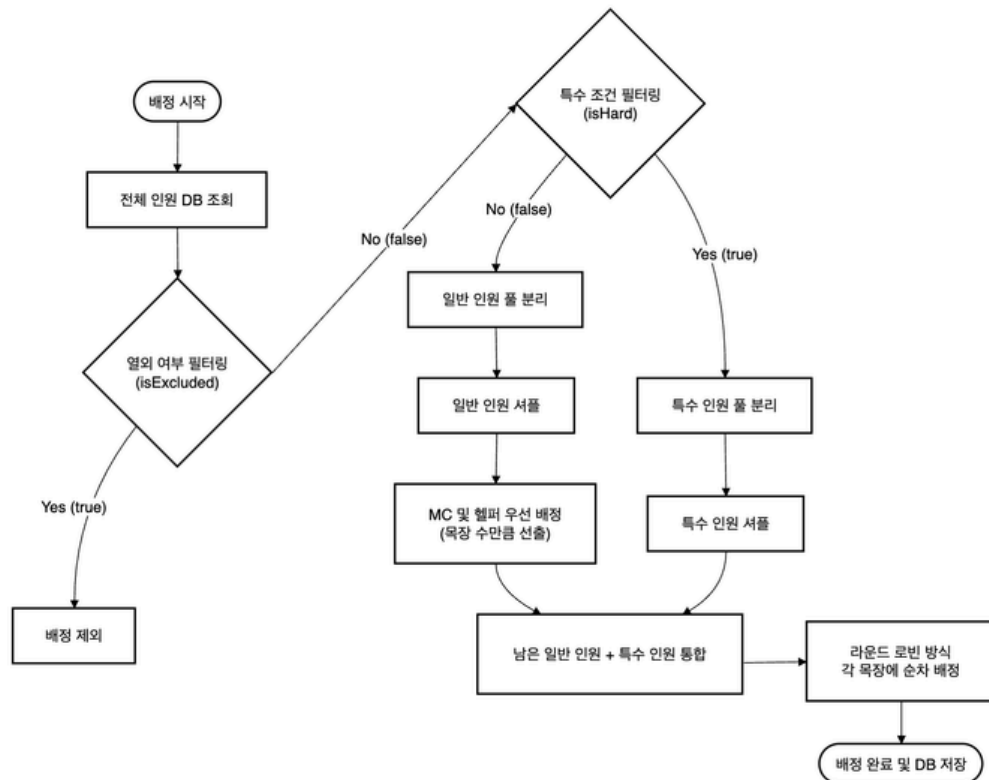
- 도입 배경 및 목적

- 기존 수동 편성 방식은 매주 관리자의 리소스가 크게 소모되며, 인적 편향이 개입될 여지가 존재
- 단순한 랜덤 분배를 넘어, '목장 모임 열외 인원 처리' 및 '특수 조건(장애 여부 등) 인원의 균등 분배' 등 복잡한 비즈니스 요구사항을 유연하게 처리할 수 있는 자동화 시스템 필요

- 핵심구현 로직

- 엔티티의 isExcluded, isHard 속성을 활용하여 전체 인원 중 배정 제외자를 1차로 필터링하고, 균등 분배가 필요한 특수 인원(장애 여부 등) 풀과 일반 인원 풀을 2차로 분리 → 이후 Collections.shuffle() 등을 적용하여 매주 겹치지 않는 새로운 조합이 나오도록 난수 기반 셔플링 적용
- 특수 인원을 먼저 각 목장에 1명씩 균등하게 배치한 후, 남은 일반 인원을 순차적으로 배분하여 특정 목장에 인원이나 조건이 편중되지 않도록 알고리즘 구현

- 알고리즘 플로우차트



- 시간 복잡도

- 전체 인원(N), 팀 수(T) 기준 $O(N + T) \approx O(N)$ 선형 시간으로 동작
- 각 멤버를 상수 번 순회하는 구조로, 인원 증가에도 안정적인 성능 유지

- 도입 효과

- 복잡한 조건이 반영된 목장 편성 작업을 클릭 한 번으로 자동화하여 관리자의 업무 시간을 획기적으로 단축
- 알고리즘 기반의 다중 필터링 및 랜덤 배정을 통해 편성의 공정성과 투명성을 기술적으로 보장, 구성원들의 서비스 신뢰도 및 사용자 경험 증대
- 추후 새로운 조건이 추가되더라도 필터링 파이프라인에 쉽게 추가할 수 있는 확장성 있는 코드 구조 구현